

Ask the Sensors

Grounded, Explainable Activity Question Answering from Wearable Signals

CS60055 Ubiquitous Computing - Hackathon Challenge 1

Technical implementation report - 10 September 2026

Team and equal work allocation

Member	Primary one-third work package
22CS30069 - Ritabrata Bharati	Data engineering: dataset audit, label mapping, raw parsing, 25 Hz resampling, dual-sensor alignment, validity masks, user-disjoint splits, and data-pipeline tests and documentation.
22CS30077 - Vishv Magarvadia	Recognition and evaluation: feature extraction, seven-class baselines and model selection, per-class metrics, confusion/strictness/robustness/overhead figures, and result verification.
23CS30041 - Pritam Mondal	QA and integration: timeline construction, Tasks 1-3, evidence grounding, guarded Task 4/Qwen path, exact-format CLI, integration tests, report assembly, and release verification.

The work is divided into three comparable end-to-end packages. Architecture review, debugging, final testing, and report review are shared equally by all three members.

Abstract

Ask the Sensors turns phone accelerometer and gyroscope recordings into answers that cite the signal intervals used to obtain them. The completed system resamples irregular raw streams to 25 Hz, constructs validity-aware windows, extracts 98 time/frequency features, recognizes seven activities, merges predictions into gap-aware intervals, and routes questions to deterministic or language-model-assisted reasoning. Evaluation is user-disjoint. On 144,056 raw dual-sensor test windows the selected 8.1 KiB balanced logistic model achieved 35.99% accuracy, 37.36% balanced accuracy, and 29.78% macro-F1. The corresponding official precomputed-feature baseline reached 54.77% accuracy and 36.21% macro-F1. Across 600 held-out questions, macro-averaged answer accuracy was 20.00%; requiring correct evidence reduced it to 9.67%. These modest results expose timeline fragmentation and class confusion rather than hiding them. All 60 automated tests pass.

Deliverables

The repository contains reproducible download/build/train/evaluate scripts, the selected model and schemas, saved results, five generated figures, a schema-conforming question-answering CLI, a small demonstration, this report source, and its reproducibly generated PDF. Raw data and Qwen weights are deliberately excluded from Git.

1. Problem Formulation and Output Contract

The challenge requires a system that receives a natural-language question and time-stamped wearable signals, then answers both what happened and which sensor evidence supports the answer. A window classifier alone is insufficient: duration, counts, onset, comparison, and grounded explanation require explicit temporal operations.

1.1 Signal and activity space

The supported input has triaxial acceleration and angular velocity. Both modalities are brought to 25 Hz in the fixed channel order Acc X, Acc Y, Acc Z, Gyro X, Gyro Y, Gyro Z. The recognition vocabulary contains exactly seven activities: lying down, sitting, standing in place, standing and moving, walking, running, and bicycling. The two standing labels are retained separately because the assignment distinguishes them and the original ExtraSensory labels provide both, even though a cleaned derived label merges them.

1.2 Four QA tiers

1. **Activity recognition:** identify an activity or verify a named activity.
2. **Temporal reasoning:** compute duration, occurrence count, first onset, and pairwise duration comparison.
3. **Evidence grounding:** attach interval, modality, channels, confidence, and signal-derived explanation to the computed answer.
4. **Open-world reasoning:** interpret behavior-level wording while refusing claims unsupported by the recognized timeline.

1.3 Exact answer schema

Every question produces the following field order. Times are seconds from the first usable retained sample, not Unix clock values.

```
Answer:                <direct answer to the query, or N/A>
Activity/Event:        <activity or event, or N/A>
Evidence:
  Timestamp(s):        <time range or ranges, or N/A>
  Sensor Modality:     <accelerometer, gyroscope, both, or N/A>
  Sensor Channel(s):   <Acc X/Y/Z, Gyro X/Y/Z, All, or N/A>
Explanation:          <reasoning grounded in the observed signal, or N/A>
```

1.4 Safety and correctness invariant

The interval used in an arithmetic result is the same interval emitted as evidence. A supported claim cannot have an empty evidence range; an inconclusive claim cannot cite a range as if it supported an answer. Low-coverage windows are discarded and low-confidence predictions become unknown. The open-world generator cannot add a timestamp, modality, or channel not contained in its supplied evidence.

1.5 Scope

This is an activity-evidence system, not a medical diagnostic device. Terms such as strenuous or unsteady are treated as observable motion patterns under declared rules. The system does not infer disease, injury, intent, or clinical state.

2. Dataset and Measured Data Characteristics

Experiments use the official ExtraSensory naturalistic context-recognition release from UC San Diego [1,2]. Sixty participants self-reported daily activities while phone and watch sensors were recorded. This setting supplies realistic placement variation and gaps, but also noisy overlapping labels and severe class imbalance.

2.1 Label audit

The original per-user labels contain 377,346 examples. Applying the seven-class, single-main-activity filter retains 307,220 labelled examples. Label counts before raw-sensor coverage filtering are 104,210 lying down; 136,356 sitting; 8,028 standing in place; 29,754 standing and moving; 22,517 walking; 1,335 running; and 5,020 bicycling. The original labels are authoritative for the two standing classes. Cleaned labels are used only as a contradiction check because their OR_standing field cannot reconstruct that distinction.

2.2 Raw archives and observed layout

The official raw accelerometer archive expands to about 29 GB and contains 377,056 session files for all 60 users. A measured iPhone example had a 33.9 Hz median rate, 23.13 s duration, and approximately 0.997 g median magnitude. A measured Android example had a 49.65 Hz rate, 16.09 s duration, and 10.02 m/s² magnitude; 23 users therefore required unit rescaling by standard gravity. The processed gyroscope archive is 9.33 GB compressed and expands to about 27 GB, with 359,912 files for 57 users. A sampled gyro session contained 800 points over 19.97 s at 40 Hz with a 0.025 s 95th-percentile gap.

2.3 Dual-sensor build

The indexed builder scans each sensor tree once and aligns files by user and example timestamp. Of 307,206 examples visited, 270,871 supplied usable dual-sensor input. Fourteen labelled examples had no indexed accelerometer file, 16,437 examples lacked a required sensor, and 19,912 were rejected for low coverage. Three users had no usable dual windows.

After resampling and 64-sample windows with a 32-sample hop, 3,535,086 of 3,792,194 candidate windows are valid. Their class distribution is shown below.

Activity	Valid windows	Share
Lying down	1,207,591	34.16%
Sitting	1,567,707	44.35%
Standing in place	94,731	2.68%
Standing and moving	335,506	9.49%
Walking	258,786	7.32%
Running	16,207	0.46%
Bicycling	54,558	1.54%

Self-report timestamps describe session-level context, not frame-perfect boundaries. Consequently, raw windows near transitions may inherit a coarse label, and missing sessions must never be interpolated into activity evidence.

3. Architecture and Preprocessing

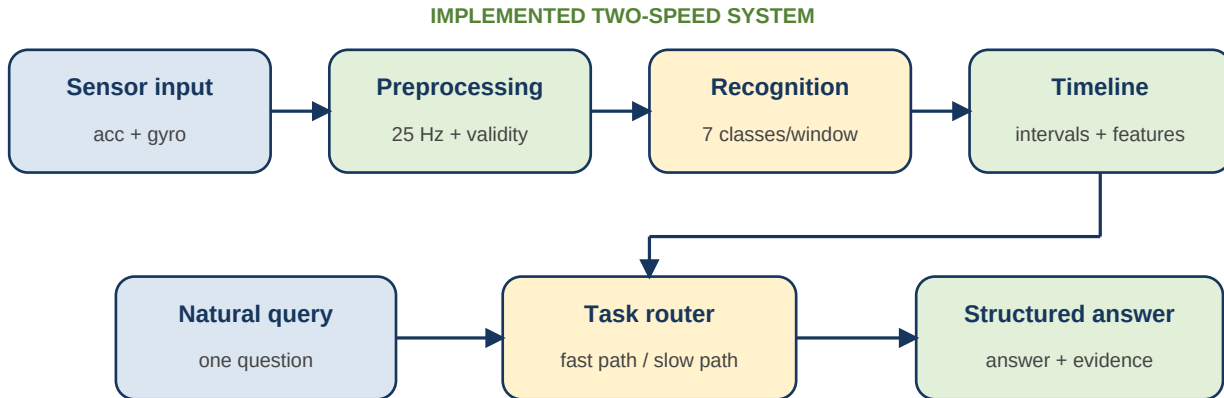


Figure 1. Implemented two-speed architecture. Recognition and timeline construction are shared; deterministic Tasks 1-3 avoid generative arithmetic, while Task 4 is schema-validated before formatting.

3.1 Raw parsing and time alignment

Readers accept sensor files containing either x, y, z with an implied time base or t, x, y, z with explicit times. Samples are sorted, duplicate timestamps removed, and non-finite or empty inputs handled without indexing failures. Each sensor stream is linearly interpolated to a 25 Hz grid, but an interpolated sample is valid only when a real observation is within 0.08 s. Combined dual-sensor validity requires both modalities. The timestamp origin is rebased to the first example that survives sensor and coverage checks, correcting the earlier label-origin defect.

3.2 Unit handling and window formation

Per-session acceleration magnitude distinguishes approximately-g inputs from approximately-m/s² inputs; the latter are divided by 9.80665. No such heuristic is applied to gyroscope values. Windows contain 64 samples (2.56 s) with 32-sample hops (1.28 s), class index, validity fraction, start/end seconds, source example timestamp, and channel matrix. This overlap increases temporal resolution while retaining enough samples for spectral features.

3.3 Reproducible split

The official fold-0 users are preserved as a 12-user recognition test set. The remaining usable users are deterministically divided into 32 training, 8 validation, and 5 QA users. Sets are mutually exclusive by UUID. Model selection sees only validation users; the 144,056 test windows are evaluated after selection. QA uses five short recordings per QA user, centered on label transitions when available, with a target of 12 consecutive example sessions per recording. Each snippet is rebased to zero to prevent multi-day wall-clock gaps from becoming false durations.

3.4 Verification

The test suite now contains **60 passing tests**. Coverage includes label filtering, empty and irregular resampling, unit normalization, multimodal alignment, missing-sensor provenance, windows, split disjointness, features, recognition artifacts, smoothing and gap-aware intervals, the exact response formatter, Tasks 1-4 rules, generated JSON rejection, evaluation metrics, build summaries, CLI integration, and report support. The synthetic generator remains a regression fixture only; no synthetic accuracy appears as a real result.

4. Seven-Class Recognition

The recognizer uses compact engineered features so the report can trace an explanation back to a modality and so CPU inference remains practical. NumPy and SciPy implement statistical and spectral operations [4,5]; scikit-learn supplies the balanced classifiers and metrics [6].

4.1 Features

The 98-dimensional dual-sensor vector includes mean, standard deviation, median, interquartile range, extrema, RMS, energy, skewness, kurtosis, zero-crossing rate, cross-axis correlations, dominant non-DC frequency, spectral entropy, and band power in 0.3-3, 3-8, and 8-12 Hz. These are computed for axes and vector magnitudes. The stored schema records feature order and modality/channel provenance.

4.2 Validation selection

All candidates use class balancing and the same capped training sample. Selection maximizes user-disjoint validation macro-F1; a candidate within one percentage point of the best is replaced by the smaller/faster alternative.

Candidate	Validation accuracy	Macro-F1	Median ms/window	Serialized size
Balanced logistic regression	38.93%	35.17%	0.00047	6.7 KiB
Compact RF, 80 trees	41.79%	34.95%	0.00173	42.5 MiB
Full RF, 300 trees	39.40%	33.30%	0.00700	1,220.9 MiB

Logistic regression is selected because it has the highest macro-F1 and is over 6,000 times smaller than the compact forest. The committed artifact, including schema metadata, is 8,276 bytes; the 1.2 GB full forest is deliberately not committed.

4.3 Untouched-user recognition result

Input/feature scope	Accuracy	Balanced accuracy	Macro-F1
Raw accelerometer + gyroscope, 98 features	35.99%	37.36%	29.78%
Raw accelerometer fallback, 49 features	34.88%	40.99%	28.13%
Official precomputed dual features, 52 retained	54.77%	not comparable in saved run	36.21%

The official-feature baseline is a useful upper reference but is not presented as the submitted raw-window pipeline: it uses the release's precomputed example features. The raw dual model improves macro-F1 over accelerometer-only but not balanced accuracy, indicating noisy gyro alignment and user/device variance.

4.4 Per-class result

Activity	Precision	Recall	F1	Test windows
Lying down	39.17%	70.10%	50.26%	31,428
Sitting	38.58%	23.94%	29.54%	36,000
Standing in place	18.53%	18.44%	18.49%	15,604
Standing and moving	25.37%	9.76%	14.09%	27,568
Walking	68.35%	46.26%	55.18%	27,958
Running	3.51%	42.59%	6.48%	958
Bicycling	26.08%	50.46%	34.38%	4,540

Rare running has only 958 test windows and is frequently confused with walking; the two standing classes are confused with sedentary classes and each other. Class weights cannot fully compensate for coarse session labels and participant shift.

5. Timeline and Deterministic Tasks 1-3

Raw probabilities are median-smoothed over neighboring windows. Predictions below 0.35 confidence are marked unknown. Consecutive windows of the same class form an `ActivityInterval` with start, end, observed duration, mean confidence, modality, supporting channels, and representative feature values. Gaps larger than the configured tolerance split intervals so missing time is not counted as observed activity.

5.1 Query router and operations

Question type	Deterministic timeline operation	Evidence returned
Identification	Activity with greatest supported duration	All intervals for selected activity
Verification	Test whether named intervals exist	Matching intervals or N/A
Duration	Sum durations of matching intervals	Exactly the summed intervals
Count	Count separated matching intervals	Every counted interval
Onset/when	Minimum matching start time	First interval
Comparison	Compare total durations for two activities	Intervals for both operands
Grounding	Restate detected event with provenance	Matching interval and feature summary

5.2 Evidence construction

Every arithmetic method returns both a value and the interval objects consumed. The formatter derives seconds, modality, and channels from those objects. Explanations name measured motion properties and mean confidence. An accelerometer-only build is labeled `accelerometer` and cannot claim gyro support; the dual build uses both and `All` only when both modalities contributed to the feature vector.

5.3 Required CLI

```
python scripts/answer_questions.py \  
  --recording recording.csv \  
  --questions questions.txt \  
  --model artifacts/raw_both_recognizer.joblib \  
  --out answers.txt --no-slm
```

The input schema is `timestamp,acc_x,acc_y,acc_z,gyro_x,gyro_y,gyro_z`. The command validates columns, normalizes time to seconds from the recording start, extracts windows and features, builds the timeline, and writes one exact-format block per question. `--no-slm` retains full Tasks 1-3 and deterministic Task 4 behavior for machines without model weights.

5.4 Example from the formatter

```
Answer: 2 intervals  
Activity/Event: walking  
Evidence:  
  Timestamp(s): 4.000 to 9.120; 12.960 to 18.080  
  Sensor Modality: both  
  Sensor Channel(s): All  
Explanation: The answer counts two separated walking intervals supported by  
  the cited dual-sensor windows and their mean confidence.
```

This block illustrates the actual output contract; it is not substituted for a measured accuracy result.

6. Task 4: Validated Local Reasoning

Task 4 uses Qwen2.5-1.5B-Instruct [3] through PyTorch and Transformers [7,8]. The 1.54-billion-parameter model is loaded locally in FP16 on the NVIDIA GPU when CUDA is available, with a CPU fallback. It receives only serialized activity intervals and feature summaries, never unrestricted raw sensor arrays.

6.1 Prompt and decoding

The prompt contains the question, a six-field JSON schema, and an EVIDENCE array. Generation is greedy (`do_sample=False`) and limited to 256 new tokens. The answer is extracted from one JSON object. There is no remote inference service and no raw wearable data leaves the machine.

6.2 Validation boundary

The deterministic validator requires answer, activity/event, timestamp pairs, modality, channels, and explanation. It checks enum membership, canonicalizes case-only enum variation, verifies every timestamp is contained in a supplied interval, checks modality compatibility, and requires claimed channels to be a subset of interval channels. Unsupported values, malformed JSON, invented time ranges, or evidence attached to `Inconclusive` raise a `GroundingError`.

The production CLI catches a validation rejection and invokes the rule layer; the rejected text never reaches the response. Rules cover prolonged lying/rest, bicycling/wheeled movement, high-intensity running/strenuous activity, and conservative unsteadiness cues. Other unsupported questions return `Inconclusive`.

6.3 Runtime validation result

The model loaded and generated successfully on the RTX 5050, but all 36 calls in the benchmark returned `Inconclusive` while also citing evidence. This violates the contract for unsupported answers, so all 36 generations were rejected. The deterministic strenuous-activity rule then returned `Yes`, citing the supplied 16.0-24.0 s running interval, modality both, channels `All`, confidence 0.880, acceleration RMS 1.820 g, and gyro RMS 0.710 rad/s. No generated claim bypassed validation.

This outcome is intentional evidence that the model cannot bypass the deterministic contract. It is distinct from the QA figure on page 8: that balanced evaluation uses deterministic Task 4 references over real recognized timelines, while the runtime benchmark isolates the optional local Qwen path.

6.4 Limitations of open-world evaluation

The 75 open-world outputs receive a fixed five-point automated audit: one point each for a non-empty structured response, correct broad answer, evidence IoU at least 0.5, exact modality/channel agreement, and an explanation containing a measured feature or interval/confidence cue. The mean is **3.56/5**. This is a deterministic reproducibility check rather than an independent human or language-model plausibility judgment. The current set covers only the disclosed rule vocabulary and paraphrases; it is not a general natural-language or clinical benchmark. No independent human panel was available, so no inter-rater agreement is claimed. Broader behavior labels require a preregistered rubric and independent annotation before accuracy can be meaningfully reported.

7. End-to-End Evaluation

Recognition metrics use the 12 untouched fold-0 users. End-to-end QA uses 25 short recordings from five additional users, with 75 questions per type and three phrasings per reference question: 600 questions total. Reference answers are computed from held-out labels using the same time support but never fed to the recognizer. Saved JSON records split/configuration hashes and all counts.

7.1 Metrics

Categorical questions use exact match. Numeric duration and onset use mean absolute error (MAE) over answered questions plus answer coverage. Temporal strictness varies the required interval IoU from 0.1 to 0.9. A grounding item is correct only if the answer matches, temporal IoU is at least 0.5, and modality/channels match. Robustness repeats evaluation at fixed random input-drop levels. These definitions penalize a correct label paired with unsupported evidence.

7.2 Results by question type

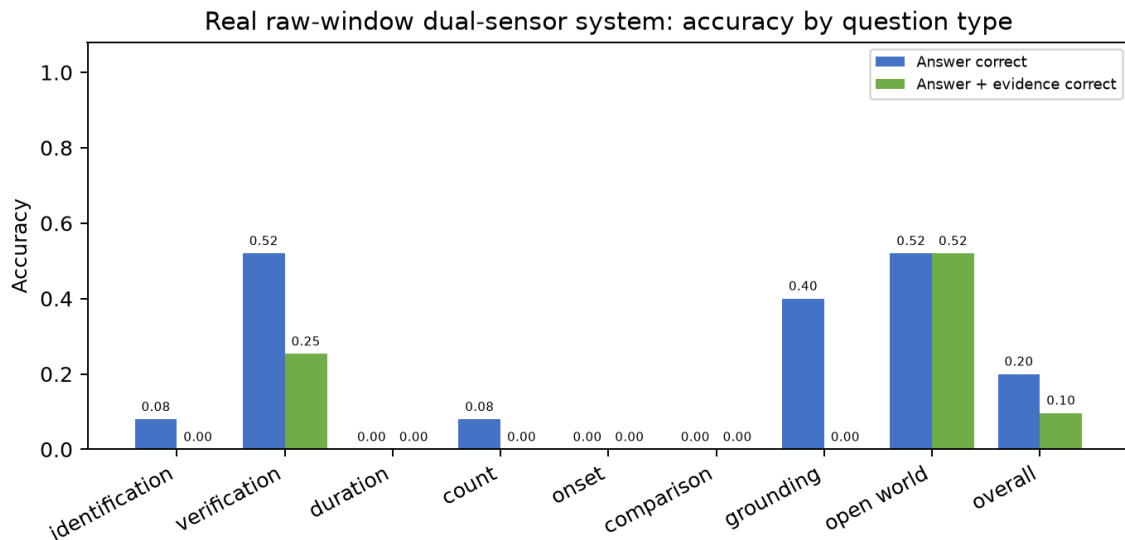


Figure 2. Plain and evidence-grounded accuracy on 75 real-data questions per type, plus macro averages. Categorical, count, and comparison answers require exact match; duration and onset require absolute error at most 2.56 s. Evidence-grounded correctness additionally requires cited-interval IoU at least 0.5 and exact modality/channel agreement.

Macro-averaging the eight plain-answer accuracies gives **20.00%**; requiring correct evidence gives **9.67%**. Plain accuracy is 8% identification, 52% verification, 0% duration, 8% count, 0% onset, 0% comparison, 40% grounding-query answer, and 52% open world. Evidence-grounded accuracy is 25.33% for verification and 52% for open world, and zero for identification, duration, count, onset, comparison, and the dedicated grounding group. For answered numeric queries, duration MAE is 108.60 s with 100% coverage, while onset MAE is 314.27 s with 26.67% coverage.

Identification macro-F1 is 12.87%, comparison macro-F1 is 0%, and open-world macro-F1 is 34.21%. Binary verification has 76.92% positive precision, 40.00% recall, 52.63% positive F1, and 76.00% specificity (20 TP, 6 FP, 19 TN, 30 FN). The poor temporal scores follow directly from fragmented and misclassified intervals.

7.3 Interpretation

The result separates software completeness from model quality: every task produces validated output, but real-data temporal reasoning is only as good as the timeline. Future work should improve calibration, transition handling, and sequence modeling rather than adding more answer templates. Reporting zeroes is important because a label-only metric would conceal this failure mode.

8. Recognition Confusion and Error Analysis

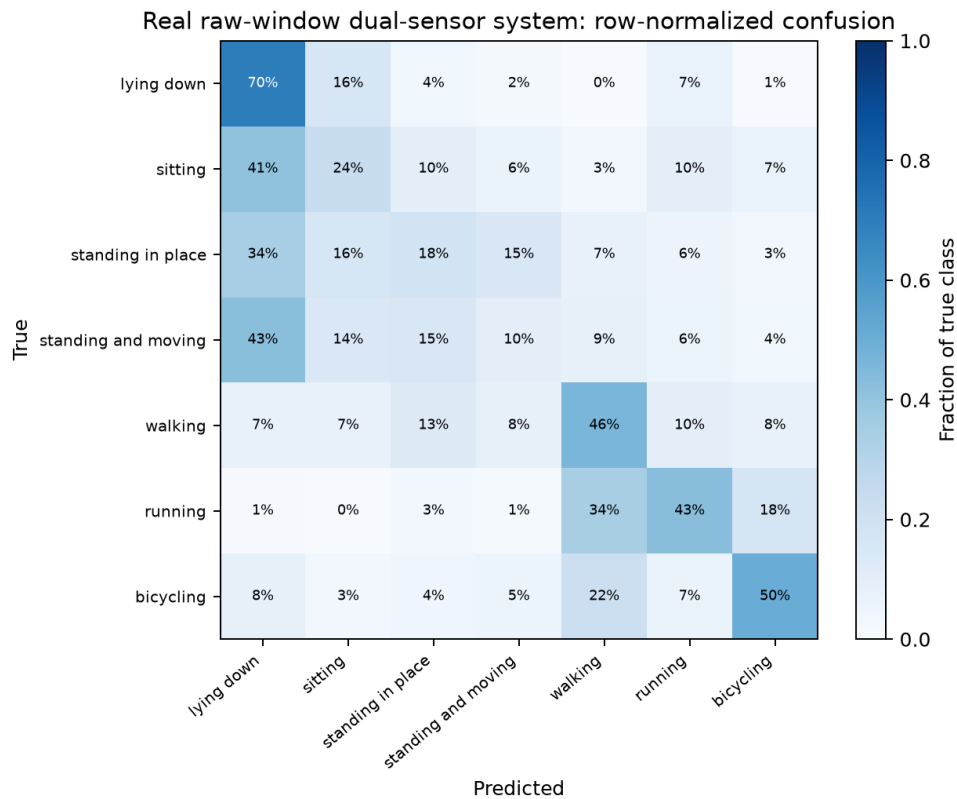


Figure 3. Row-normalized confusion matrix for 144,056 untouched-user raw dual-sensor windows. Percentages are fractions of each true class; exact counts are saved in the evaluation JSON.

Lying down is the strongest sedentary class at about 70% recall. Sitting is predicted as lying in 41% of its windows. Standing in place and standing/moving have only about 18% and 10% recall, respectively, with a large bias toward lying. Walking reaches 46% recall, running 43%, and bicycling 50%, but running has little support and a low F1 because many other moving windows are also predicted as running.

The errors have three plausible, non-exclusive sources. First, self-reported labels apply to approximately 20-second sessions and need not align with every 2.56-second window. Second, the phone coordinate frame varies across users and placements; per-axis features are therefore only partly invariant. Third, the selected linear model trades nonlinear capacity for an 8 KiB artifact and low latency. The forests increase validation accuracy but do not improve macro-F1 enough to justify their size under the selection rule.

The confusion also explains the QA pattern. A false sedentary interval may dominate identification, extra short intervals change counts, and a single boundary shift invalidates strict grounding or onset even if the broad activity is reasonable. A stronger sequence model or orientation-normalized representation is a higher-value improvement than relaxing the scoring threshold.

9. Strictness and Accuracy-Overhead Trade-off

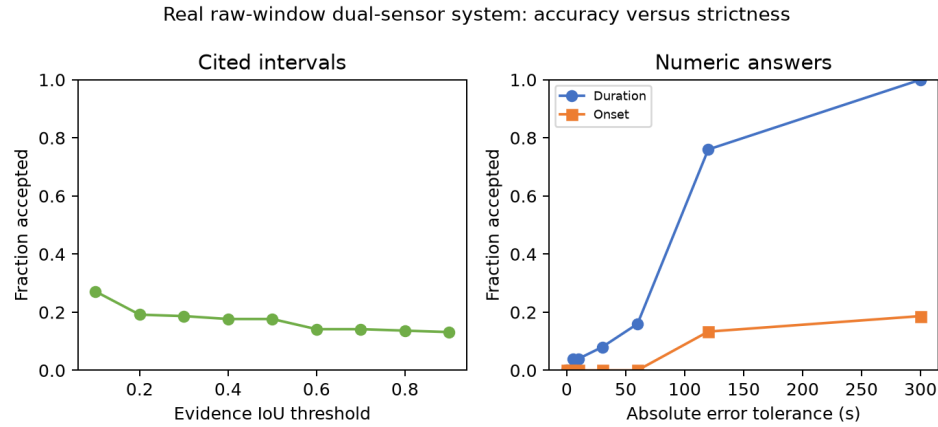


Figure 4. Strictness analysis. Left: fraction of cited answers accepted as the evidence-IoU threshold increases from 0.1 to 0.9. Right: duration/onset acceptance as absolute numeric tolerance increases; unanswered numeric queries count as incorrect.

Evidence acceptance falls from 27.17% at IoU 0.1 to 13.17% at IoU 0.9; the 0.5 value is 17.67%. Duration acceptance is 0% at the primary 2.56 s rule, 16% at 60 s, 76% at 120 s, and 100% at 300 s. Onset remains 0% through 60 s, reaches 13.33% at 120 s, and 18.67% at 300 s. The curves distinguish near misses from large timeline errors.

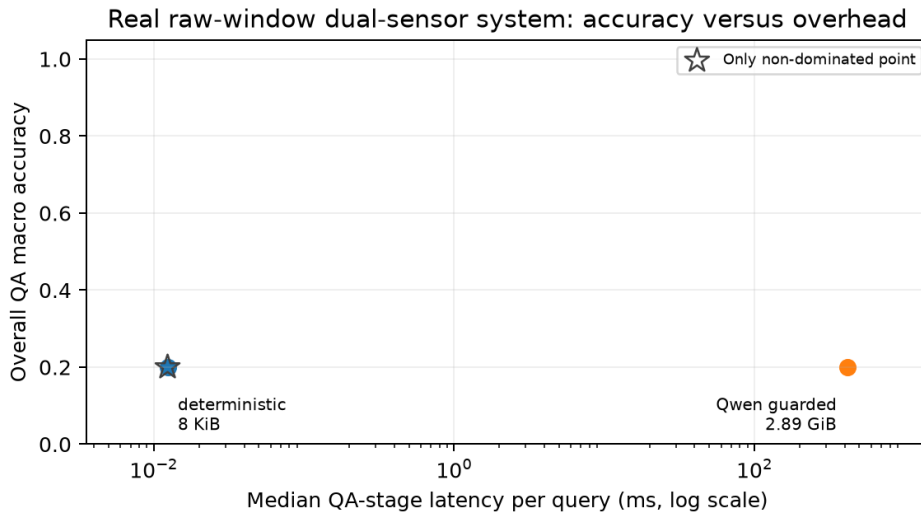


Figure 5. Overall QA macro accuracy versus median QA-stage latency per query. Deterministic latency is measured per question; Qwen-guarded latency is workload-weighted across seven deterministic tiers and one Task 4 tier. Labels include committed model/cache size; the star marks the only non-dominated configuration.

Both configurations reach 20.00% overall QA accuracy because every sampled Qwen generation was rejected and safely fell back to the deterministic rule. Deterministic QA takes 0.0124 ms/query with an 8 KiB recognizer. The Qwen-guarded mixed workload takes 414.265 ms/query and adds a 2.89 GiB model cache without an accuracy gain; it is therefore dominated. This negative result is a measured accuracy-overhead tradeoff, not an edge-efficiency claim.

10. Robustness and Efficiency

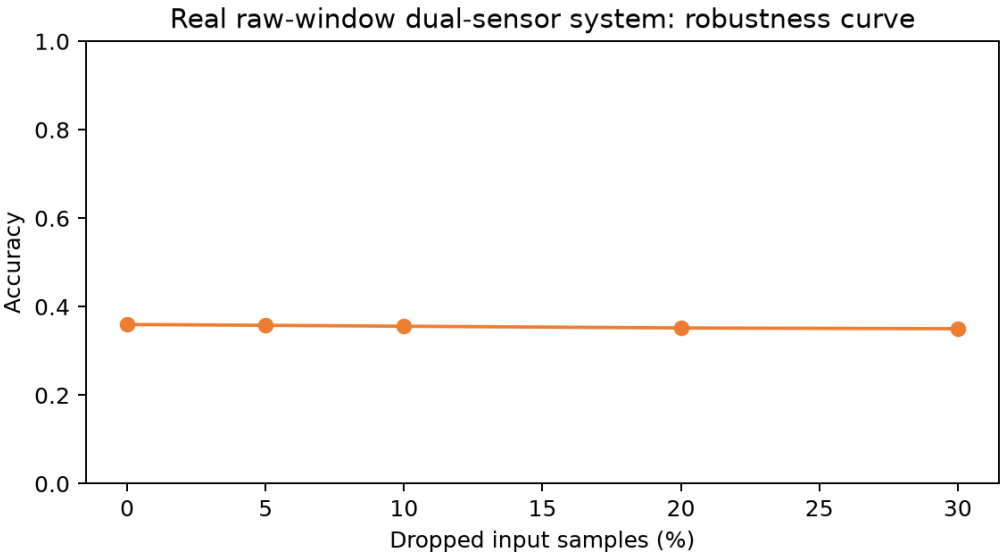


Figure 6. Recognition accuracy under deterministic random removal of input samples, averaged over three repeats at each level.

Accuracy changes from 35.99% with no additional drops to 35.80%, 35.59%, 35.20%, and 35.03% at 5%, 10%, 20%, and 30% removal. The gradual 0.96-point loss at 30% reflects validity-aware interpolation and aggregate features. It does not imply immunity to long contiguous gaps; windows below the coverage threshold are excluded instead of guessed.

10.1 Measured deterministic path

Measurements used Python 3.14.4 under WSL2 on an AMD Ryzen 9 8940HX-class machine with 32 logical processors. Each timing has five warm-ups and 30 measured runs.

Operation	Batch	Median	p95	Peak traced memory
Model probability prediction	256 windows	0.567 ms	0.658 ms	0.227 MiB
Raw window to features and probabilities	256 windows	9.288 ms	9.572 ms	2.091 MiB
Deterministic Tasks 1-3	4 questions	0.049 ms	0.086 ms	0.004 MiB

The selected committed model is 8,276 bytes and represents 693 learned coefficients/intercepts. The raw-to-probability measurement is the realistic recognition cost because it includes 98-feature extraction; the smaller model-only number demonstrates that feature computation dominates. Process CPU utilization during these short serial batches was approximately 3.1% of total machine capacity, about one fully occupied logical processor.

10.2 Task 4 cost

The optional Qwen path is deliberately benchmarked separately because it loads a multi-billion-byte model and performs autoregressive decoding, whereas Tasks 1-3 are sub-millisecond deterministic operations. Qwen has 1,543,714,304 parameters and occupies 3,098,955,668 cache bytes (2.89 GiB). FP16 loading took 110.16 s. Across five warm-ups and 30 measured validated-pipeline runs, median latency was 3,314.03 ms and p95 was 3,548.79 ms; peak allocated GPU memory was 3,002.43 MiB and final process RSS was 2,034.05 MiB. All generations were rejected and safely routed to the deterministic fallback, so these timings measure the conservative SLM-plus-validation path, not successful generative answer quality.

11. Reproducibility, Limitations, and References

11.1 Reproduction commands

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.lock
pip install -e .
pytest -q

python scripts/build_dataset.py --raw-root /path/to/extrasensory \
    --out /path/to/processed_both --modalities both
python scripts/train_recognizer.py --processed /path/to/processed_both \
    --splits artifacts/raw_both_split_manifest.json \
    --model artifacts/raw_both_recognizer.joblib \
    --results artifacts/raw_both_recognition_results.json
python scripts/evaluate_system.py --processed /path/to/processed_both \
    --splits artifacts/raw_both_split_manifest.json \
    --model artifacts/raw_both_recognizer.joblib \
    --recognition-results artifacts/raw_both_recognition_results.json \
    --efficiency artifacts/raw_both_efficiency.json \
    --slm-efficiency artifacts/slm_efficiency.json \
    --out artifacts/raw_both_evaluation_results.json
python scripts/generate_figures.py \
    --results artifacts/raw_both_evaluation_results.json \
    --out report/figures/real_raw_both
python report/build_report.py
python report/check_report.py
```

Task 4 was tested with PyTorch 2.14.0+cu130, Transformers 5.17.0, Accelerate 1.15.0, and psutil 7.2.2. The core pipeline remains usable without these optional packages through `--no-slm`. Result JSON files store configuration and split hashes; model metadata stores feature and class order. Raw archives, expanded data, the 1.2 GB experimental forest, virtual environments, and Qwen weights are excluded from Git.

11.2 Limitations and next work

Recognition and temporal QA are not yet competitive: participant shift, orientation variance, coarse labels, severe imbalance, and linear decision boundaries remain. QA references are generated from labels rather than independently annotated question/answer pairs. Robustness uses random sample removal, not burst loss. The open-world test vocabulary is narrow and has no human rubric panel. The next technical priorities are orientation-invariant features, probability calibration, sequence-aware recognition, transition-level annotations, class-aware thresholding, and an external paraphrase/grounding set.

11.3 Academic integrity and AI-use disclosure

Claude (Anthropic) assisted with the initial repository scaffold, configuration, preprocessing modules, synthetic generator, and early tests. OpenAI Codex assisted with assignment audit, planning, implementation, data-pipeline repairs, testing, experiment orchestration, result verification, and report generation. Ritabrata Bharati, Vishv Magarvadia, and Pritam Mondal share responsibility for reviewing the work and the submitted artifact. AI-generated text or code was never treated as experimental evidence; reported measurements come from committed scripts and saved outputs. A chronological disclosure is retained in `docs/AI_USE_LOG.md`.

11.4 References

- [1] ExtraSensory Dataset, UC San Diego. <http://extrasensory.ucsd.edu/>
- [2] Y. Vaizman, K. Ellis, and G. Lanckriet. ExtraSensory naturalistic context-recognition dataset and study. *IEEE Pervasive Computing*, 16(4), 2017.
- [3] Qwen Team. Qwen2.5-1.5B-Instruct model card. <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>
- [4] C. R. Harris et al. Array programming with NumPy. *Nature*, 585, 2020. <https://doi.org/10.1038/s41586-020-2649-2>
- [5] P. Virtanen et al. SciPy 1.0. *Nature Methods*, 17, 2020. <https://doi.org/10.1038/s41592-019-0686-2>
- [6] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR*, 12, 2011.
- [7] A. Paszke et al. PyTorch. *NeurIPS*, 2019. [8] T. Wolf et al. Transformers. *EMNLP System Demonstrations*, 2020.